



Assignment 2

Migrating to Laravel MVC

IS333 Web-Based Information Systems – Spring 2026

Team Size: Same group as assignment 1

Deadline: Friday 15-05-2026 @ 11:00 PM

Overview

In this assignment, you will migrate the Single-Page Application (SPA) you built in Assignment 1 into a full Laravel MVC project. Your application should expose the same features as the original SPA — database operations, client-side and server-side validation, and the third-party API integration — but now structured according to Laravel conventions. You will also write automated tests to verify that the core functionality of your application works correctly.

Requirements

1. Laravel MVC Architecture

- **Port your application.** Recreate all features from Assignment 1 inside a Laravel project, following the MVC pattern: Models for database interaction, Controllers for business logic, and Blade views for the UI.
- **Database migrations.** Define your database schema using Laravel migrations rather than raw SQL and implement your queries.
- **Data persistence.** All create and read operations from Assignment 1 must continue to function, storing and retrieving data from the MySQL database.
- **Routes.** Register all application routes in routes/web.php or routes/api.php as appropriate.

2. Validations

- **Server-side validation.** You Must Use Laravel's built-in validation rules (e.g., required, email, max, unique) inside your controllers or Form Request classes. Return meaningful error messages to the view.
- **Client-side validation.** Retain the JavaScript validation logic from Assignment 1.

3. Laravel Master Layout

- **Blade layout file.** Create a master Blade layout (e.g., layouts/app.blade.php) that defines the overall page structure including the <head>, header, and footer sections.
- **Header & Footer.** The header and footer from Assignment 1 must be incorporated as Blade includes.



4. Third-Party API Integration

The same API you integrated in Assignment 1 must continue to function within the Laravel project.

- **Controller-based API calls.** Move any PHP-side API calls into a dedicated controller method or a service class.
- **Error handling.** Handle API failures gracefully and display a user-friendly message in the Blade view if the API is unavailable.
- **API keys.** Store your API keys in the .env file and access them via config() or env().

5. Automated Laravel Tests

Write at least three automated test functions within the Laravel testing framework (PHPUnit / Pest). At least one test must be a Feature test.

- **Feature test (required).** Tests an HTTP endpoint end-to-end — for example, submitting a form, verifying a redirect or a JSON response, and asserting that a record was created in the database.
- **Unit tests.** Test isolated logic such as validation rules, helper methods, or model scopes.
- **Running tests.** All tests must pass when running php artisan test on a fresh installation.

Submission

- **Team members file.** Include a plain-text file **Team_Members.txt** containing team number, and each member's full name and student ID.
- **Database backup.** Make sure your “database.sqlite” file is present under “database” folder.
- **Delete** the “vendor” folder before compressing your project.
- **Compressed archive.** Place all files in a single zip / rar folder named: YourTeamNumber_**ASSIGNMENT-2** (e.g., **15_ASSIGNMENT-2.zip**).
- **Upload to the Classroom submission link** posted by the instructor before the deadline.

⚠ Submissions that do not follow the naming convention or are missing the database backup will receive a 1-mark deduction.



Good Practices Checklist

Before submitting, verify that your project meets the following standards:

Criterion
Laravel MVC architecture: application ported to MVC structure, with data correctly stored in the database
Client-side and server-side validations correctly migrated to the Laravel project
Laravel Master Layout applied: header and footer included in the welcome page via Blade templates
Third-party API integrated and functional within the Laravel project
Automated Laravel tests: at least three test functions, including at least one feature test